

Java Backend Interview Series

Sub-Chapter 1.11: Multithreading Fundamentals

Thread Lifecycle | Synchronization | Locks | Memory Model | Deadlock

52 Q&A | Advanced Questions | Exercises | Cheat Sheet

1. Thread Creation, Lifecycle & States

A Java thread progresses through well-defined states managed by the JVM scheduler. Threads can be created by extending **Thread**, implementing **Runnable**, implementing **Callable** (returns a value), or using an **ExecutorService**. The preferred approach is Runnable/Callable submitted to an executor rather than raw Thread.

```
// 1. Extending Thread (discouraged – couples task with execution mechanism)
class MyThread extends Thread {
    public void run() { System.out.println("Thread: " + getName()); }
}

// 2. Implementing Runnable (preferred for fire-and-forget tasks)
Runnable task = () -> System.out.println("Runnable on " + Thread.currentThread().getName());
Thread t = new Thread(task, "worker-1");
t.start(); // never call run() directly – that executes on the current thread

// 3. Callable + Future (for tasks that return values or throw checked exceptions)
Callable<Integer> compute = () -> {
    Thread.sleep(100);
    return 42;
};
ExecutorService exec = Executors.newFixedThreadPool(4);
Future<Integer> future = exec.submit(compute);
Integer result = future.get(); // blocks until result ready

// Thread state inspection
Thread.State state = t.getState();
// NEW -> RUNNABLE -> (BLOCKED | WAITING | TIMED_WAITING) -> TERMINATED
```

State	Meaning	How to Enter
NEW	Created but start() not called	new Thread(...)
RUNNABLE	Running or ready to run (OS-scheduled)	t.start()
BLOCKED	Waiting to acquire a monitor lock	Contended synchronized block
WAITING	Indefinitely waiting for notification	Object.wait(), LockSupport.park()
TIMED_WAITING	Waiting with a timeout	Thread.sleep(), wait(ms), join(ms)
TERMINATED	run() has completed	Normal return or uncaught exception

Daemon Threads

Daemon threads (t.setDaemon(true) before start()) do not prevent JVM shutdown. The JVM exits when all non-daemon threads finish, abandoning any running daemon threads. Used for background housekeeping (GC, finalizer thread). Never use daemon threads for I/O or critical tasks that must complete before shutdown.

2. synchronized Keyword & Intrinsic (Monitor) Locks

Every Java object has an associated **intrinsic lock** (monitor). The **synchronized** keyword acquires this lock, providing mutual exclusion and establishing a happens-before relationship. It can be applied to instance methods, static methods (locks the Class object), and arbitrary blocks.

```
public class Counter {
    private int count = 0;

    // synchronized instance method – locks 'this'
    public synchronized void increment() { count++; }
    public synchronized int get() { return count; }

    // synchronized block – locks a specific object (more granular)
    private final Object lock = new Object();
```

```

public void decrement() {
    synchronized (lock) { count--; }
}

// synchronized static method – locks Counter.class
private static int instances = 0;
public static synchronized void registerInstance() { instances++; }
}

// wait / notify – must be called inside synchronized block
class BoundedBuffer<T> {
    private final Queue<T> buf = new LinkedList<>();
    private final int cap;
    BoundedBuffer(int cap) { this.cap = cap; }

    public synchronized void put(T item) throws InterruptedException {
        while (buf.size() == cap) wait(); // releases lock while waiting
        buf.add(item);
        notifyAll(); // wake consumers
    }

    public synchronized T take() throws InterruptedException {
        while (buf.isEmpty()) wait();
        T item = buf.poll();
        notifyAll(); // wake producers
        return item;
    }
}

```

wait() in a loop — always use while, not if

Always use `while(condition) { wait(); }` never `if(condition) { wait(); }`. Reason: spurious wakeups — a thread may wake from `wait()` without being notified. Also, between `wait()` returning and re-acquiring the lock, another thread may change the condition. The while loop re-checks the condition after every wakeup.

3. volatile Keyword & Java Memory Model (JMM)

The **Java Memory Model (JMM)** defines how threads interact through memory. Each thread may cache variables in its local working memory. **volatile** guarantees visibility (reads/writes go directly to main memory) and prevents instruction reordering around the volatile access. It does NOT guarantee atomicity of compound operations like `i++`.

```

// volatile – visibility guarantee
class StopFlag {
    private volatile boolean stopped = false; // without volatile, loop may run forever
    public void stop() { stopped = true; }
    public void run() {
        while (!stopped) { /* do work */ }
    }
}

// volatile does NOT make compound ops atomic
volatile int counter = 0;
counter++; // NOT atomic: read -> increment -> write (race condition)
// Use AtomicInteger instead

// JMM Happens-Before (HB) rules:
// 1. Program order: each action HB later actions in same thread
// 2. Monitor lock: unlock HB subsequent lock of same monitor
// 3. volatile write HB subsequent volatile read of same variable
// 4. Thread start: Thread.start() HB any action in the started thread
// 5. Thread join: all actions in thread T HB Thread.join(T) return
// 6. Transitivity: if A HB B and B HB C then A HB C

```

```
// Double-Checked Locking – MUST use volatile
class Singleton {
    private static volatile Singleton instance; // volatile essential!
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Guarantee	synchronized	volatile	AtomicXxx
Visibility	Yes	Yes	Yes
Atomicity	Yes (block)	No (single read/write only)	Yes (CAS ops)
Ordering	Full fence	Partial fence	Partial fence
Reentrancy	Yes	N/A	N/A
Performance	Slower (blocking)	Faster (no blocking)	Fast (lock-free)

4. Thread Coordination: join, interrupt & ThreadGroup

Threads coordinate their lifecycle using **join()** (wait for a thread to finish), **interrupt()** (signal a thread to stop cooperatively), and **Thread.sleep()** / **yield()**. Proper interruption handling is critical for clean shutdown.

```
// join() – wait for thread completion
Thread worker = new Thread(() -> heavyCompute());
worker.start();
worker.join(); // current thread blocks until worker finishes
worker.join(5000); // with timeout (ms)

// Interruption – cooperative cancellation protocol
class CancellableTask implements Runnable {
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                doWork();
                Thread.sleep(100); // throws InterruptedException if interrupted
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); // RESTORE the interrupted flag!
                break; // clean exit
            }
        }
        cleanup();
    }
}

// Interrupting the thread
Thread t = new Thread(new CancellableTask());
t.start();
t.interrupt(); // sets interrupted flag; sleeping/waiting threads get InterruptedException

// Thread.sleep vs Object.wait
Thread.sleep(ms); // timed wait, holds locks, InterruptedException on interrupt
obj.wait(ms); // releases lock, wakes on notify/timeout/interrupt

// yield() – hint to scheduler (not guaranteed)
Thread.yield(); // current thread willing to yield processor to equal-priority threads
```

Interrupted Flag Semantics

`Thread.interrupted()` is static: tests AND CLEARS the interrupted flag of the current thread.

`Thread.currentThread().isInterrupted()` is instance: tests WITHOUT clearing the flag. `InterruptedException` clears the flag upon being thrown. Always restore the flag after catching `InterruptedException` if you cannot propagate the exception.

5. Deadlock, Livelock & Starvation

A **deadlock** occurs when two or more threads each hold a resource the other needs. A **livelock** is when threads keep responding to each other without making progress. **Starvation** occurs when a thread cannot access resources due to other threads perpetually taking priority.

```
// Classic deadlock: two threads locking in opposite order
Object lockA = new Object();
Object lockB = new Object();
Thread t1 = new Thread(() -> {
    synchronized (lockA) {
        Thread.sleep(50);
        synchronized (lockB) { /* work */ } // waits for lockB
    }
});
Thread t2 = new Thread(() -> {
    synchronized (lockB) {
        Thread.sleep(50);
        synchronized (lockA) { /* work */ } // waits for lockA
    }
});
// t1 holds lockA, wants lockB; t2 holds lockB, wants lockA -> DEADLOCK
// Prevention: always acquire locks in a consistent global order
// Or use tryLock() with timeout
ReentrantLock lockX = new ReentrantLock();
ReentrantLock lockY = new ReentrantLock();
if (lockX.tryLock(100, MILLISECONDS)) {
    try {
        if (lockY.tryLock(100, MILLISECONDS)) {
            try { /* do work */ }
            finally { lockY.unlock(); }
        }
    } finally { lockX.unlock(); }
}

// Detection: thread dumps (jstack / jcmd <pid> Thread.print)
// JVM detects monitor deadlocks and reports in thread dumps
// ThreadMXBean: ManagementFactory.getThreadMXBean().findDeadlockedThreads()
```

Problem	Description	Prevention
Deadlock	Circular lock dependency — threads blocked forever	Consistent lock ordering, tryLock with timeout, lock-free algorithms
Livelock	Threads actively changing state but no progress (e.g., backing off together)	Randomised back-off, priority adjustment
Starvation	Low-priority thread never gets CPU/lock	Fair locks (<code>ReentrantLock(true)</code>), priority-aware scheduling

6. Thread-Safety Patterns & Immutability

The safest way to avoid concurrency bugs is to eliminate shared mutable state. **Immutable objects** are inherently thread-safe. **Thread confinement** ensures only one thread accesses an object. **ThreadLocal** provides per-thread storage. When sharing is necessary, use proper synchronization or lock-free data structures.

```
// Immutable class – always thread-safe
@Immutable
public final class Money {
    private final BigDecimal amount;
    private final String currency;
    public Money(BigDecimal amount, String currency) {
        this.amount = Objects.requireNonNull(amount);
        this.currency = Objects.requireNonNull(currency);
    }
    public BigDecimal getAmount() { return amount; }
    public String getCurrency() { return currency; }
    public Money add(Money other) { return new Money(amount.add(other.amount), currency); }
}

// ThreadLocal – per-thread storage (no sharing needed)
private static final ThreadLocal<SimpleDateFormat> DATE_FORMAT =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
public String formatDate(Date d) {
    return DATE_FORMAT.get().format(d); // each thread gets its own SDF instance
}
// IMPORTANT: call remove() when done (e.g., in Servlet filter finally block)
// to prevent memory leaks in thread pool scenarios
// Safe publication – how to share an object safely
// 1. Static initialiser (JVM guarantees thread-safe class loading)
public static final Config INSTANCE = new Config();
// 2. volatile field
private volatile Config config;
// 3. Properly locked field
private Config config;
public synchronized Config getConfig() { return config; }
// 4. Effectively immutable object shared after construction
// (use safe publication + no further mutation)
```

Rules for Immutable Classes

1. Declare the class final (prevent subclassing). 2. All fields private final. 3. No setter methods. 4. Perform defensive copies of mutable inputs in the constructor. 5. Return defensive copies from getters that return mutable types (arrays, collections). Strings, Integer, LocalDate, BigDecimal are all immutable by design.

7. Thread Priority, Daemon Threads & Uncaught Exceptions

Java threads have integer priorities (1–10, default 5). The JVM maps these to OS thread priorities, but the actual scheduling behaviour is OS-dependent and priorities are unreliable for correctness. **UncaughtExceptionHandler** lets you handle exceptions that escape a thread's run() method.

```
// Thread priority (use sparingly – OS-dependent)
Thread t = new Thread(task);
t.setPriority(Thread.MAX_PRIORITY); // 10
t.setPriority(Thread.NORM_PRIORITY); // 5 (default)
t.setPriority(Thread.MIN_PRIORITY); // 1

// Daemon thread – JVM can exit while daemon still running
Thread housekeeping = new Thread(() -> { while(true) { monitor(); sleep(1000); } });
housekeeping.setDaemon(true); // must be set BEFORE start()
```

```

housekeeping.start();

// UncaughtExceptionHandler – catch exceptions that escape run()
Thread.setDefaultUncaughtExceptionHandler((thread, ex) -> {
    log.error("Uncaught exception in thread {}", thread.getName(), ex);
    metrics.increment("thread.uncaught.exceptions");
});

// Per-thread handler
t.setUncaughtExceptionHandler((thread, ex) -> {
    System.err.println("Thread " + thread.getName() + " failed: " + ex.getMessage());
});

// Thread naming – essential for debugging
ThreadFactory namedFactory = r -> {
    Thread th = new Thread(r);
    th.setName("payment-processor-" + threadCounter.incrementAndGet());
    th.setDaemon(false);
    th.setUncaughtExceptionHandler(globalHandler);
    return th;
};

```

8. Thread Dump Analysis, Diagnostics & Best Practices

Thread dumps are the primary tool for diagnosing deadlocks, hangs, and performance issues. They can be captured via **jstack**, **jcmd**, **kill -3**, or programmatically via **ThreadMXBean**. Understanding the output is a key interview skill.

```

// Programmatic thread dump
ThreadMXBean tmx = ManagementFactory.getThreadMXBean();

// Find deadlocked threads
long[] deadlocked = tmx.findDeadlockedThreads();
if (deadlocked != null) {
    ThreadInfo[] infos = tmx.getThreadInfo(deadlocked, true, true);
    for (ThreadInfo info : infos) {
        System.out.println(info.toString());
    }
}

// Full thread dump
for (ThreadInfo ti : tmx.dumpAllThreads(true, true)) {
    System.out.println(ti);
}

// CPU profiling per thread
long cpuTime = tmx.getThreadCpuTime(Thread.currentThread().getId()); // nanoseconds

// jstack / jcmd commands (from terminal)
// jstack <pid> -> thread dump to stdout
// jcmd <pid> Thread.print -> same via jcmd
// kill -3 <pid> -> on Linux/Mac, dumps to stderr
// jcmd <pid> VM.native_memory -> native memory tracking
// Java Flight Recorder (JFR):
// jcmd <pid> JFR.start duration=60s filename=rec.jfr
// jfr print --events jdk.ThreadSleep rec.jfr

```

Best Practice	Rationale
Prefer Executor over raw Thread	Decouples task from execution, handles lifecycle
Name your threads/pools	Invaluable in thread dumps and monitoring
Always unlock in finally	Prevents lock leaks on exception
Minimise lock scope	Reduces contention; hold locks for shortest time possible

Use lock ordering	Prevents deadlocks; document the order
Prefer immutability	Eliminates shared mutable state entirely
Test with stress tools	Use jcstress, ConTest, or Thread Weaver for concurrency tests

Interview Q&A; - Standard Questions

Q1. What are the different ways to create a thread in Java?

Four main approaches: (1) **Extend Thread** — override `run()`; limited because Java is single-inheritance. (2) **Implement Runnable** — implement `run()`, pass to Thread constructor; preferred for tasks. (3) **Implement Callable<V>** — like Runnable but returns a value and can throw checked exceptions; submitted to `ExecutorService`, returns `Future<V>`. (4) **Lambda/method reference** — Runnable `r = () -> doWork()`; Thread `t = new Thread(r)`. Best practice: use `ExecutorService.submit(callable)` over raw threads.

Q2. What is the difference between `start()` and `run()` on a Thread?

`start()` creates a new OS thread and schedules the `run()` method to execute on it — the calling thread continues concurrently. **`run()`** called directly executes the runnable in the *current* thread synchronously — no new thread is created. It is a common bug to call `run()` instead of `start()`. `start()` can only be called once; calling it again throws `IllegalThreadStateException`.

Q3. Explain the Java thread states (`Thread.State` enum).

NEW: created, `start()` not called. **RUNNABLE**: executing or eligible to execute on CPU. **BLOCKED**: waiting to enter a synchronized block/method held by another thread. **WAITING**: indefinitely waiting — `Object.wait()`, `Thread.join()`, `LockSupport.park()`. **TIMED_WAITING**: waiting with timeout — `Thread.sleep(n)`, `wait(n)`, `join(n)`, `parkNanos()`. **TERMINATED**: `run()` completed normally or via uncaught exception. A thread cannot transition back to NEW or be restarted.

Q4. What is the difference between synchronized method and synchronized block?

A **synchronized method** locks 'this' (instance method) or the Class object (static method) for the entire method duration. A **synchronized block** locks any specified object for a scoped section of code. Blocks offer: (1) finer granularity — lock only what's needed; (2) ability to use a private lock object (avoids locking 'this' which clients could also lock); (3) shorter lock hold time — reduces contention.

Q5. What is the volatile keyword and when should you use it?

volatile ensures: (1) **Visibility** — writes are immediately visible to all threads (no CPU cache reads); (2) **Ordering** — prevents instruction reordering around the volatile access (a write-before and read-after fence). Use cases: status/stop flags, safe publication of immutable objects, double-checked locking singleton. Do NOT use for: compound operations like `i++` (use `AtomicInteger`), operations spanning multiple variables (use `synchronized`).

Q6. What is a race condition? Give an example.

A race condition occurs when the correctness of a program depends on the relative timing or interleaving of threads. Example: two threads executing 'count++' simultaneously. `count++` expands to: read count, increment, write back. If both threads read 0 concurrently, both write 1 — the counter is 1 instead of 2. Fix: `synchronize` the method, use `AtomicInteger.incrementAndGet()`, or use a lock.

Q7. What is a deadlock and what are the four necessary conditions?

Deadlock is a state where threads are permanently blocked waiting for resources held by each other. Coffman's four necessary conditions: (1) **Mutual Exclusion** — resources cannot be shared; (2) **Hold and Wait** — thread holds resource while waiting for another; (3) **No Preemption** — resources cannot be forcibly taken; (4) **Circular Wait** — circular chain of threads each waiting on the next. Prevention: eliminate any one condition (e.g., impose a global lock order to break circular wait).

Q8. How does `wait()` differ from `sleep()`?

`wait()`: must be called inside `synchronized` block; **releases the monitor lock** while waiting; woken by `notify()`/`notifyAll()` or timeout; throws `InterruptedException`. **`sleep()`**: static method on `Thread`; does **NOT release any lock**; just pauses the thread; throws `InterruptedException`. Summary: use `wait/notify` for inter-thread communication/condition synchronization; use `sleep` for simple timed pauses.

Q9. What is thread starvation and how do you prevent it?

Starvation occurs when a thread is perpetually denied CPU time or lock access because higher-priority threads or unfair scheduling always take precedence. Prevention: (1) Use **fair locks** — `ReentrantLock(true)` uses FIFO ordering; (2) Avoid indefinitely holding locks; (3) Use equal priorities unless there is a strong reason for differentiation; (4) Use thread pools with fair work queues (`LinkedBlockingQueue` vs `SynchronousQueue`).

Q10. What is the happens-before relationship in the JMM?

Happens-before (HB) is the JMM's formal guarantee that actions in one thread are visible to another. Key rules: (1) **Program order** within a thread; (2) **Monitor lock**: unlock HB next lock of same monitor; (3) **volatile write** HB subsequent volatile read; (4) **Thread.start()** HB any action in the new thread; (5) **Thread.join()**: all actions in thread T HB `join()` return. Without HB, the compiler/CPU may reorder or cache operations.

Q11. What is an immutable class and how do you create one?

An immutable class cannot be modified after construction. Rules: (1) Declare class **final**; (2) All fields **private final**; (3) **No setters**; (4) **Defensive copy** of mutable inputs in constructor; (5) **Defensive copy** of mutable return values in getters. Examples: `String`, `Integer`, `LocalDate`, `BigDecimal`. Immutable objects are inherently thread-safe — no synchronization needed.

Q12. What is ThreadLocal and when should you use it?

`ThreadLocal` provides a per-thread variable — each thread has its own isolated copy. Common uses: `SimpleDateFormat` (not thread-safe), database `Connection`, user session context in web servers. Usage: `ThreadLocal.withInitial(() -> new SimpleDateFormat())`. Critical: **always call remove()** after use in thread pool contexts to prevent memory leaks (thread pools reuse threads, so `ThreadLocal` state persists across tasks).

Q13. What is the difference between notify() and notifyAll()?

notify() wakes up a single arbitrary thread waiting on the object's monitor. This can cause **missed signals** — if the wrong thread is woken (one that can't proceed), the system can livelock. **notifyAll()** wakes all waiting threads — they all re-check the condition and one proceeds. Safer but less efficient. Rule of thumb: use `notifyAll()` unless you have a single condition and all waiting threads are interchangeable.

Q14. How can you detect a deadlock programmatically?

Use **ThreadMXBean**: `ManagementFactory.getThreadMXBean().findDeadlockedThreads()` returns IDs of threads involved in deadlocks (monitor or ownable-synchronizer deadlocks). Then call `getThreadInfo(ids, true, true)` for full stack traces. From command line: `jstack <pid>` prints thread dump including deadlock report. JFR event `jdk.JavaMonitorDeadlock` also captures deadlocks. Set up periodic detection with a background thread in production services.

Q15. What is a daemon thread?

A daemon thread is a background thread that does not prevent JVM shutdown. The JVM exits when all non-daemon (user) threads have finished, regardless of running daemons. Set with **`t.setDaemon(true)`** before `start()`. Examples: GC thread, finalizer thread, JIT compiler thread. Never use daemon threads for tasks requiring guaranteed completion (I/O commits, DB transactions) as they can be abruptly terminated.

Q16. What does Thread.interrupt() actually do?

`Thread.interrupt()` sets the thread's **interrupted flag**. If the thread is blocking in `sleep()`, `wait()`, or `join()`, it immediately throws **`InterruptedException`** (and clears the flag). If the thread is running normally, only the flag is set — the thread must check `isInterrupted()` periodically. Key: always restore the flag after catching `InterruptedException`: `Thread.currentThread().interrupt()` — this allows callers to detect the interruption.

Q17. What is the double-checked locking pattern and why does it need volatile?

DCL lazily initialises a singleton while avoiding synchronization on every call: `'if (instance == null) { synchronized(Singleton.class) { if (instance == null) instance = new Singleton(); } }'`. Without **volatile** on the instance field, the JIT can reorder the constructor steps: a partially constructed object could be published — another thread sees a non-null instance before it is fully initialised. **volatile** prevents this reordering via a write barrier.

Q18. What is a livelock?

A livelock occurs when threads are not blocked but keep changing state in response to each other without making progress. Example: two threads back off and retry when they detect conflict — if they always back off at the same time they keep colliding. Analogous to two people in a corridor both stepping aside in the same direction repeatedly. Prevention: randomised back-off (e.g., exponential backoff with jitter, as in Ethernet CSMA/CD).

Q19. What is the meaning of 'safe publication' of an object?

Safe publication means making a constructed object visible to other threads in a way that guarantees they see a fully initialised object. Mechanisms: (1) Static initialiser (class loading is synchronised by JVM); (2) **volatile** field; (3) **final** field (all final fields are guaranteed visible after constructor, per JMM); (4) Synchronized accessor; (5) Concurrent collection (e.g., `ConcurrentHashMap.put/get` have HB guarantees). Without safe publication, threads may see a partially constructed object.

Q20. What is spurious wakeup and how do you handle it?

A spurious wakeup is when a thread wakes from `Object.wait()` or `Condition.await()` without being notified, interrupted, or timed out. The POSIX standard permits this and the JVM does not prevent it. Defence: always use a while loop to re-check the condition: `'while (!conditionMet) { wait(); }'` — not `'if (!conditionMet) { wait(); }'`. This pattern is the standard producer-consumer idiom.

Q21. How does `Thread.sleep()` interact with locks?

`Thread.sleep()` does **NOT release any held locks**. A sleeping thread still holds all monitors and ReentrantLocks it acquired. This is a common bug: sleeping while holding a lock blocks other threads from acquiring it, reducing concurrency and potentially causing starvation. By contrast, `Object.wait()` releases the monitor lock while waiting. Use `wait(ms)` inside synchronized blocks when you need to release the lock during a timed wait.

Q22. What is thread confinement and what are its forms?

Thread confinement means an object is accessible only from one thread — eliminating the need for synchronization. Forms: (1) **Ad-hoc confinement** — by convention, only one thread uses the object; fragile. (2) **Stack confinement** — local variables confined to method stack (most common). (3) **ThreadLocal** — JVM-enforced per-thread storage. Swing/AWT components are thread-confined to the Event Dispatch Thread (EDT). Document thread ownership in Javadoc with `@NotThreadSafe` or `@GuardedBy`.

Q23. What information does a thread dump provide?

A thread dump shows: (1) All thread names, IDs, states, and priorities; (2) Full stack trace of each thread; (3) Locks held and locks waited on; (4) Deadlock reports (if any); (5) Blocked threads and which thread holds the contended lock. Key states to look for: **BLOCKED** (lock contention), **WAITING** (wait/park), **TIMED_WAITING** (sleep/wait with timeout). Take multiple dumps 10s apart to distinguish stuck threads from slow ones.

Q24. What is the difference between `Thread.interrupted()` and `isInterrupted()`?

`Thread.interrupted()` is a static method that: tests the interrupted flag of the *current* thread AND clears the flag. `isInterrupted()` is an instance method that: tests the flag of the thread it's called on WITHOUT clearing it. Use `isInterrupted()` in `run()` loops to check for interruption without consuming the flag. Use `interrupted()` when you want to consume and act on the interruption in a single operation.

Q25. What happens when an exception escapes a thread's run() method?

If an unchecked exception escapes `run()`, the thread terminates. If an `UncaughtExceptionHandler` is set (on the thread or globally via `Thread.setDefaultUncaughtExceptionHandler`), its `uncaughtException(Thread, Throwable)` method is called. Otherwise (no handler), the stack trace is printed to `System.err` and the thread dies silently. With `ExecutorService.submit()`, the exception is wrapped in the `Future` and thrown when `Future.get()` is called as an `ExecutionException`.

Q26. What is a thread-safe class?

A class is **thread-safe** if it behaves correctly when accessed from multiple threads simultaneously, without requiring any additional synchronization from the client. Techniques: synchronization, immutability, thread confinement, atomic variables, concurrent data structures. Not all classes need to be thread-safe — it is perfectly valid to document a class as `@NotThreadSafe` and require callers to synchronize externally. `Vector/Hashtable` are thread-safe but replaced by synchronized wrappers and concurrent collections.

Interview Q&A; - Advanced Questions

[ADV] Q27. Explain instruction reordering and how the JMM prevents unsafe reordering.

CPUs and compilers reorder instructions for performance (e.g., store buffers, out-of-order execution). The JMM defines a **partial order** — actions within a thread appear ordered (as-if-serial), but across threads, only happens-before guarantees ordering. **Memory barriers** (`LoadLoad`, `StoreStore`, `LoadStore`, `StoreLoad`) are inserted by the JVM at volatile accesses, synchronized entry/exit, and final field writes. A `StoreLoad` barrier (the most expensive) is inserted after a volatile write and before a volatile read, flushing the store buffer and invalidating caches.

[ADV] Q28. What is the ABA problem in lock-free programming?

The ABA problem occurs in CAS (Compare-And-Swap) operations: thread T1 reads value A, is suspended; T2 changes A to B then back to A; T1 resumes, CAS succeeds (sees A) but the structure has changed in a semantically important way (e.g., a node was removed and reinserted). Fix: use **AtomicStampedReference** (adds a version stamp) or **AtomicMarkableReference**. In Java's `ConcurrentLinkedQueue`, ABA is avoided by never reusing nodes.

[ADV] Q29. How does ReentrantLock's fair mode work internally?

`ReentrantLock(true)` uses `AbstractQueuedSynchronizer` (AQS) with a FIFO CLH queue. On `lock()`, if the lock is free but there is a non-empty queue, the thread enqueues itself rather than acquiring immediately — this prevents barging. `tryAcquire()` checks `hasQueuedPredecessors()` to enforce ordering. Fair mode reduces throughput (~30x in benchmarks) due to context switches and cache misses, but eliminates starvation. Non-fair mode (default) allows barging — threads that just ran may reacquire before sleeping threads wake up.

[ADV] Q30. What is AbstractQueuedSynchronizer (AQS) and how is it used?

AQS is the foundation for most `java.util.concurrent` synchronizers: `ReentrantLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `ReentrantReadWriteLock`. It manages a volatile `int` state and a CLH (Craig-Landin-Hagersten) queue of waiting threads. Subclasses implement `tryAcquire/tryRelease` (exclusive) or `tryAcquireShared/tryReleaseShared`. AQS handles queueing, parking (`LockSupport.park()`), and unparking (`LockSupport.unpark()`). The state field semantics are defined by each synchronizer (e.g., lock count for `ReentrantLock`, permits for `Semaphore`).

[ADV] Q31. Explain the visibility guarantees of final fields after construction.

The JMM provides a special guarantee for **final fields**: once a constructor completes, any thread that obtains a reference to the object is guaranteed to see the correct values of all final fields — even without synchronization. This is implemented via a freeze action (a `StoreStore` barrier after the final field write). Caveat: the reference to the object must not escape the constructor (via 'this' escape). This is why immutable classes with all-final fields are safely shareable without volatile.

[ADV] Q32. What is a false sharing problem in multithreaded code?

False sharing occurs when two threads write to different variables that happen to reside on the same CPU cache line (typically 64 bytes). When one thread writes, it invalidates the entire cache line — forcing the other thread to reload it, even though its variable wasn't changed. Fix: pad fields to separate cache lines using `@Contended` (JVM flag: `-XX:-RestrictContended`) or manual padding. Example: `LongAdder/LongAccumulator` use `@Contended` internally for this reason. Profiling tool: Linux `perf`, JFR `jdk.CPUCache` events.

[ADV] Q33. How does Java's memory model handle publication via final fields?

The JMM's **final field freeze** action guarantees that all writes to final fields in the constructor are visible to any thread that reads the object reference, provided the object reference is not published before construction is complete (no `this`-escape). This means an immutable object constructed properly can be safely published via a data race on a non-volatile/unsynchronized reference — a special JMM exception. Non-final fields of an immutable-looking object do NOT get this guarantee.

[ADV] Q34. What is the significance of the happens-before rule for `Thread.start()`?

The HB rule for `Thread.start()` states: **all actions performed before `Thread.start()` happen-before any action in the started thread**. This means any data written to shared fields before calling `t.start()` is guaranteed to be visible to the new thread — no additional synchronization needed. Conversely, the HB rule for `Thread.join()` guarantees that all actions in the joined thread happen-before the return from `join()` in the joining thread.

[ADV] Q35. How does `LockSupport.park/unpark` work compared to `wait/notify`?

`LockSupport` provides low-level thread blocking primitives underlying AQS. **`park()`** disables the current thread (returns immediately if a permit is available, else blocks). **`unpark(thread)`** grants a permit to a specific thread (wakes it if parked, or causes next `park()` to return immediately). Advantages over `wait/notify`: (1) no monitor required; (2) `unpark` before `park` works correctly (no missed signal); (3) targets a specific thread not 'one random waiter'. Used by AQS internally to suspend/resume queued threads.

[ADV] Q36. What is a memory visibility bug and how would you find one?

A memory visibility bug occurs when changes made by one thread are not seen by another due to CPU caching or compiler optimisation. Symptoms: infinite loops (flag not updated), stale cached values, `ConcurrentModificationException`. Tools: (1) `jctstress` (Java Concurrency Stress) - stress-tests memory model guarantees; (2) thread sanitizers; (3) code review for missing volatile/synchronization. JMM says: without HB, a read may see any value previously written — including stale values. Fix: volatile, synchronized, or atomic variables.

[ADV] Q37. Explain the CLH queue used in `ReentrantLock/AQS`.

The CLH (Craig-Landin-Hagersten) queue is a non-blocking queue of nodes, each representing a waiting thread. Each node spin-waits on the predecessor's status rather than a central lock — reducing cache coherence traffic. In AQS's adapted version: (1) Threads create a `Node` and CAS-enqueue it; (2) The head node holds the current lock owner; (3) When the lock is released, the head's successor is unparked; (4) Threads park (not spin) while waiting — reduces CPU waste. The queue supports both exclusive and shared modes (for `ReadWriteLock`).

[ADV] Q38. What are the different types of memory barriers?

Memory barriers prevent CPU/compiler reordering: **`LoadLoad`**: ensures loads before barrier complete before loads after; **`StoreStore`**: ensures stores before barrier are flushed before stores after; **`LoadStore`**: ensures loads before barrier complete before stores after; **`StoreLoad`**: full barrier — ensures all stores/loads before are visible before loads/stores after. `StoreLoad` is the most expensive (mfence on x86). Volatile read inserts `LoadLoad+LoadStore`; volatile write inserts `StoreStore+StoreLoad`.

[ADV] Q39. How do you properly stop a thread that is blocked in I/O?

`Thread.interrupt()` works for `java.nio` channels (which implement `InterruptibleChannel`) — blocking I/O throws `ClosedByInterruptException` and closes the channel. For `java.io` streams (`InputStream.read()`), `interrupt()` does NOT unblock the call. Workarounds: (1) close the underlying socket/stream from another thread; (2) use NIO non-blocking I/O with `Selector`; (3) use a watchdog thread that closes the socket after a timeout. Always design for cancellation when writing long-running I/O code.

[ADV] Q40. What is the producer-consumer pattern and how do you implement it thread-safely?

The producer-consumer pattern decouples data production from consumption via a shared buffer. Thread-safe implementations: (1) **BlockingQueue** — `put()` blocks when full, `take()` blocks when empty; `LinkedBlockingQueue` or `ArrayBlockingQueue`. (2) **synchronized + wait/notifyAll** — manual `BoundedBuffer`. (3) **Disruptor** — lock-free ring buffer for ultra-low latency. `BlockingQueue` is the standard choice: it handles all wait/notify internally, supports bounded and unbounded variants, and is highly optimised.

[ADV] Q41. Explain happens-before with an example of a data race.

Data race: two threads access a shared variable with no HB between them, and at least one is a write. Example: Thread A writes `flag = true`; Thread B reads `flag`. Without HB, B may read false even after A writes true. Fix: make `flag` volatile (volatile write HB volatile read). An important subtlety: HB is about the order of operations across threads, not wall-clock time. Even if A's write completes at time T_1 and B's read starts at $T_2 > T_1$, without HB the read can still observe the old value.

[ADV] Q42. What is the difference between a lock and a CAS operation?

A **lock** (mutex/monitor) is a blocking mechanism — a thread trying to acquire a held lock is suspended (context switch overhead). Lock-based code is straightforward but can cause deadlock, priority inversion, and convoying. **CAS (Compare-And-Swap)** is a single CPU instruction (`LOCK CMPXCHG` on x86) that atomically reads, compares, and conditionally writes. Non-blocking — threads spin on failure. CAS-based algorithms (lock-free) avoid blocking overhead but require careful handling of ABA, retry loops, and can cause starvation under high contention.

[ADV] Q43. How does the JVM handle thread-local allocation buffers (TLABs) in the context of multithreading?

TLABs are regions of the Eden heap pre-allocated to individual threads. Object allocation within a TLAB is thread-local (bump pointer) — no synchronization needed. When a TLAB is exhausted, the thread requests a new one from the JVM (which requires a global allocation lock but is rare). TLABs eliminate allocation contention almost entirely in multi-threaded programs. Tuning: `-XX:TLABSize`, `-XX:+PrintTLAB`. Large objects bypass TLABs and allocate directly in the old/Humongous region (G1).

[ADV] Q44. What is priority inversion and how can it be prevented?

Priority inversion occurs when a high-priority thread is blocked waiting for a resource held by a low-priority thread, and a medium-priority thread preempts the low-priority one — effectively the medium-priority thread runs before the high-priority thread. Real-world example: Mars Pathfinder mission (1997) suffered a priority inversion. Solutions: (1) **Priority inheritance** — temporarily elevate the low-priority thread's priority (OS-level); (2) **Priority ceiling** — lock holder runs at the max priority of any thread that could lock it; (3) Avoid priority-based scheduling for correctness.

[ADV] Q45. What is compare-and-swap (CAS) and how does AtomicInteger use it?

`CAS(addr, expected, newValue)` atomically: reads `addr`, if it equals `expected` writes `newValue` and returns true, else returns false. `AtomicInteger.compareAndSet(expected, update)` uses the `sun.misc.Unsafe.compareAndSwapInt` native call mapping to a `LOCK CMPXCHG` instruction on x86. `AtomicInteger.incrementAndGet()` implements a spin loop: `'int current; do { current = get(); } while (!compareAndSet(current, current+1)); return current+1;'`. In Java 9+, `VarHandle.compareAndSet()` replaces `Unsafe` for public API.

[ADV] Q46. How does synchronized interact with the Java Memory Model at the bytecode level?

Synchronized blocks are compiled to **monitorenter** and **monitorexit** bytecodes. **monitorenter**: acquires the object's monitor; inserts LoadLoad+LoadStore barriers — ensures fresh reads after lock acquisition. **monitorexit**: releases the monitor; inserts StoreStore+StoreLoad barriers — flushes all writes before release. This is why synchronization provides both mutual exclusion AND memory visibility. The JIT compiles these to native mutex operations (or lock-free fastpath via biased locking in older JVMs).

[ADV] Q47. What is thread-local storage overhead and when can it become a problem?

ThreadLocal stores values in a ThreadLocalMap on each Thread object — a hash map keyed by ThreadLocal instances (with WeakKey references). Overhead per access: hash computation + open-addressing probe. Issues: (1) **Memory leak**: if a ThreadLocal is garbage-collected (key becomes null), the value is orphaned in the map until the thread dies or another get()/set() triggers cleanup. (2) **Virtual threads**: millions of virtual threads each with a ThreadLocalMap can exhaust heap. Solution: Scoped Values (Java 21) for virtual thread-friendly per-task context.

[ADV] Q48. Describe a scenario where using too many threads degrades performance.

Excessive threads cause: (1) **Context switch overhead** — saving/restoring CPU registers, cache invalidation; each switch costs ~1-10 microseconds. (2) **Memory** — each platform thread stack is ~512KB-1MB; 10,000 threads = 5-10GB stack memory. (3) **Lock contention** — more threads competing for shared resources increases wait time. (4) **Scheduler thrashing** — OS scheduler overhead grows with thread count. Optimal thread count for CPU-bound: $N+1$ (N = CPU cores). For I/O-bound: $N * (1 + \text{Wait/Compute})$ — or use virtual threads.

[ADV] Q49. What is the Java Memory Model's treatment of 64-bit long/double fields without synchronization?

The JMM permits JVMs to treat reads/writes of 64-bit long and double fields as two separate 32-bit operations (word tearing). Without synchronization, a thread could read the high 32 bits from one write and the low 32 bits from another — resulting in a corrupt value. In practice, all modern 64-bit JVMs (HotSpot included) perform atomic 64-bit operations, but the spec allows non-atomicity on 32-bit JVMs. Solution: declare long/double fields volatile or access them under synchronization.

[ADV] Q50. How does biased locking work in HotSpot and why was it removed?

Biased locking (removed in Java 21) optimised the case where a monitor is accessed by only one thread. The object header stores the locking thread's ID — lock acquisition by the same thread requires no CAS or atomic operation (just a pointer comparison). On contention by a second thread, the JVM revokes the bias (a stop-the-world operation). Revocation overhead outweighed benefits in modern multi-threaded applications. Removed in Java 17 (deprecated) and fully removed in Java 21. Lightweight locking (CAS) is now the fastpath.

[ADV] Q51. Explain the connection between the Java Memory Model and CPU cache coherence protocols.

Modern CPUs use MESI (Modified-Exclusive-Shared-Invalid) protocol to keep per-core caches coherent. A cache line in 'Modified' state is dirty and owned by one core — other cores must request it. The JMM's happens-before rules map to CPU instructions that trigger cache coherence: volatile write inserts mfence (full barrier, forces store buffer flush and cache invalidation). On x86, Total Store Order (TSO) means loads are generally coherent — only StoreLoad needs an explicit barrier. ARM/POWER have weaker models requiring more explicit barriers.

[ADV] Q52. What is a wait-free algorithm vs lock-free, and can Java achieve them?

Lock-free: at least one thread makes progress in a finite number of steps (others may starve). **Wait-free**: every thread completes its operation in a bounded number of steps (no starvation). Java examples: Lock-free: AtomicInteger (CAS retry loop — one thread progresses, others may retry). Wait-free in practice: reads of volatile variables, final fields. True wait-free algorithms (Kogan-Petrunkin queue) exist but are complex. In production Java, 'lock-free' typically means CAS-based with acceptable retry probability under realistic contention.

Coding Exercises

Exercise 1: Thread-Safe Bounded Counter with wait/notifyAll

Implement a BoundedCounter that blocks on increment() when at max and on decrement() when at 0. Use synchronized + wait/notifyAll.

```
public class BoundedCounter {
    private int count = 0;
    private final int max;
    public BoundedCounter(int max) { this.max = max; }
    public synchronized void increment() throws InterruptedException {
        while (count == max) wait();
        count++;
        notifyAll();
    }
    public synchronized void decrement() throws InterruptedException {
        while (count == 0) wait();
        count--;
        notifyAll();
    }
    public synchronized int get() { return count; }
}
```

Exercise 2: Deadlock-Free Bank Transfer

Implement transfer(from, to, amount) that acquires two account locks without deadlock by using a consistent global ordering (by account ID).

```
public class BankAccount {
    private final int id;
    private double balance;
    public BankAccount(int id, double balance) { this.id = id; this.balance = balance; }
    public int getId() { return id; }
    public synchronized double getBalance() { return balance; }
    public synchronized void debit(double amount) { balance -= amount; }
    public synchronized void credit(double amount) { balance += amount; }
}

public class TransferService {
    public void transfer(BankAccount a, BankAccount b, double amount) {
        // Always lock lower ID first to prevent circular wait
        BankAccount first = a.getId() < b.getId() ? a : b;
        BankAccount second = a.getId() < b.getId() ? b : a;
        synchronized (first) {
            synchronized (second) {
                if (a.getBalance() < amount) throw new IllegalStateException("Insufficient funds");
                a.debit(amount);
                b.credit(amount);
            }
        }
    }
}
```

Exercise 3: Custom ThreadLocal DateFormatter

```
public class ThreadSafeDateParser {
    private static final ThreadLocal<SimpleDateFormat> SDF =
        ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
    public static Date parse(String dateStr) throws ParseException {
        return SDF.get().parse(dateStr); // each thread uses its own SDF
    }
}
```



```
}  
public static String format(Date date) {  
    return SDF.get().format(date);  
}  
// IMPORTANT: in thread pool scenarios, clean up to prevent memory leaks  
public static void cleanup() {  
    SDF.remove();  
}  
}
```

Exercise 4: Immutable Value Object

```
public final class Coordinate {  
    private final double latitude;  
    private final double longitude;  
    private final int[] rawData; // defensive copy of mutable input  
    public Coordinate(double lat, double lon, int[] rawData) {  
        if (lat < -90 || lat > 90) throw new IllegalArgumentException("Invalid latitude");  
        if (lon < -180 || lon > 180) throw new IllegalArgumentException("Invalid longitude");  
        this.latitude = lat;  
        this.longitude = lon;  
        this.rawData = Arrays.copyOf(rawData, rawData.length); // defensive copy  
    }  
    public double getLatitude() { return latitude; }  
    public double getLongitude() { return longitude; }  
    public int[] getRawData() { return Arrays.copyOf(rawData, rawData.length); } // defensive copy  
    public Coordinate translate(double dLat, double dLon) {  
        return new Coordinate(latitude + dLat, longitude + dLon, rawData);  
    }  
}
```

Debug & Fix Scenarios

Debug 1: Missed Notification (if instead of while)

```
// BUG: using if instead of while - vulnerable to spurious wakeups
public synchronized void put(Object item) throws InterruptedException {
    if (buffer.size() == capacity) wait(); // BUG: should be while
    buffer.add(item);
    notify();
}

// Fix: always use while loop for wait conditions
public synchronized void put(Object item) throws InterruptedException {
    while (buffer.size() == capacity) wait(); // FIXED
    buffer.add(item);
    notifyAll(); // use notifyAll to wake all waiting consumers
}
```

Debug 2: ThreadLocal Memory Leak in Thread Pool

```
// BUG: ThreadLocal not cleaned up in Servlet (thread pool reuses threads)
public class RequestFilter implements Filter {
    private static final ThreadLocal<User> CURRENT_USER = new ThreadLocal<>();
    public void doFilter(Request req, Response res, FilterChain chain) throws Exception {
        CURRENT_USER.set(authenticateUser(req));
        chain.doFilter(req, res);
        // BUG: no finally block -- if chain throws, CURRENT_USER is never removed
        // Next request on this thread sees previous user's data!
    }

    // Fix: always use try/finally
    public void doFilterFixed(Request req, Response res, FilterChain chain) throws Exception {
        try {
            CURRENT_USER.set(authenticateUser(req));
            chain.doFilter(req, res);
        } finally {
            CURRENT_USER.remove(); // FIXED: always clean up
        }
    }
}
```

Debug 3: Interrupted Flag Not Restored

```
// BUG: swallowing InterruptedException without restoring flag
public void run() {
    while (true) {
        try {
            process(queue.take());
        } catch (InterruptedException e) {
            // BUG: silently swallowed -- interrupted flag is now cleared
            // Caller has no way to know this thread was interrupted
            log.warn("Interrupted");
        }
    }
}

// Fix: restore the interrupted flag and exit cleanly
public void run() {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            process(queue.take());
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); // FIXED: restore flag
        }
    }
}
```

```
break; // exit the loop
}
}
log.info("Worker shutting down cleanly");
}
```

Debug 4: Visibility Bug - Non-volatile Stop Flag

```
// BUG: stopped flag may be cached by thread, loop runs forever
public class Worker implements Runnable {
    private boolean stopped = false; // BUG: not volatile
    public void stop() { stopped = true; }
    public void run() {
        while (!stopped) { // may see stale cached value
            doWork();
        }
    }
}

// Fix: declare stopped volatile
public class WorkerFixed implements Runnable {
    private volatile boolean stopped = false; // FIXED: volatile ensures visibility
    public void stop() { stopped = true; }
    public void run() { while (!stopped) { doWork(); } }
}
```

Multiple Choice Questions

Q1. Which thread state is entered when a thread calls `Object.wait()`?

- A) BLOCKED
- B) SLEEPING
- C) WAITING [Correct]
- D) PARKED

Q2. What does `Thread.interrupted()` do that `isInterrupted()` does not?

- A) Checks a different thread
- B) Clears the interrupted flag [Correct]
- C) Throws `InterruptedException`
- D) Restarts the thread

Q3. What is the minimum number of Coffman conditions required for deadlock?

- A) 2
- B) 3
- C) 4 [Correct]
- D) 1

Q4. Which of these does NOT release a monitor lock?

- A) `Object.wait()`
- B) `Thread.sleep()` [Correct]
- C) `Object.wait(1000)`
- D) Both A and C release it

Q5. Why must double-checked locking use a volatile field?

- A) To ensure mutual exclusion
- B) To prevent constructor reordering visible to other threads [Correct]
- C) To allow multiple threads to enter the synchronized block
- D) To make the field serializable

Q6. What happens when you call `start()` on an already-started thread?

- A) It restarts the thread
- B) It throws `IllegalThreadStateException` [Correct]
- C) It is silently ignored
- D) It throws `IllegalStateException`

Quick Reference Cheat Sheet

Concept	Key Detail	Gotcha
Thread.start()	Creates new OS thread, schedules run()	run() executes on current thread -- no new thread
Thread States	NEW-RUNNABLE-BLOCKED-WAITING-TIMED_WAITING-TERMINATED	BLOCKED=waiting for lock; WAITING=wait/join/park
synchronized method	Locks 'this' (instance) or Class (static)	Holds lock even during Thread.sleep()
synchronized block	Locks specified object, finer granularity	Prefer private lock object over 'this'
volatile	Visibility + ordering; NO compound-op atomicity	i++ is NOT atomic even with volatile
wait()	Releases lock, waits for notify; needs synchronized	Always use while(!cond) wait(); not if
notify()	Wakes one random waiter	Use notifyAll() unless 100% sure of single condition
sleep()	Timed pause; does NOT release locks	InterruptedException clears interrupted flag
interrupt()	Sets interrupted flag; wakes blocked threads	Always restore flag: Thread.currentThread().interrupt()
join()	Current thread waits for target to finish	join() returns immediately if target already terminated
ThreadLocal	Per-thread storage; isolates non-thread-safe objects	Call remove() in thread pools to prevent memory leak
Immutable class	final class, final fields, defensive copies	This-escape in constructor breaks final-field guarantee
Deadlock	Circular lock dependency; threads blocked forever	Prevent: consistent lock ordering or tryLock+timeout
Livelock	Threads active but no progress	Fix: randomised backoff
Starvation	Thread perpetually denied resource	Fix: fair locks ReentrantLock(true)
DCL Singleton	volatile + double null-check + synchronized	volatile prevents partial-construction visibility
JMM HB rules	unlock->lock, volatile-write->volatile-read, start->run, run->join	Without HB, reads may see stale values
False sharing	Different variables, same cache line -- invalidates each other	Fix: @Contended or manual padding to 64 bytes